

Introdução às APIs

Programação para a Web 2

Aula 07

Evandro J.R. Silva

O que é uma API?

- É *Application Programming Interface*?
 - Sim, mas mais do que o nome transparece.
 - É uma **interface** que um programa apresenta a outros programas e a humanos.
 - Apesar de terem sido projetadas para trabalhar com outros programas, elas são destinadas principalmente a serem **compreendidas e utilizadas** pelos **humanos** que escrevem esses outros programas.
 - São os blocos de construção que permitem a interoperabilidade das principais plataformas de negócios da web.

Por que precisamos de APIs?

- As APIs surgiram da necessidade de trocar informações com fornecedores de dados que estão capacitados para resolver problemas específicos
 - Assim, as pessoas das outras empresas não precisam gastar tempo resolvendo esses problemas por conta própria.
- Não precisamos reinventar/reimplementar, por exemplo
 - Google maps;
 - Sistemas de pagamento;
 - Vinculação de contas.

Por que precisamos de APIs?

- As APIs permitem que novos produtos sejam desenvolvidos rapidamente.
- As novas empresas/*startups* podem focar no diferencial de seu produto enquanto utilizam diversas soluções prontas.

Casos de uso de APIs

- APIs para desenvolvedores internos primeiro, e externos depois
 - A abertura da API interna pode trazer várias vantagens:
 - Ecossistema de desenvolvimento;
 - Nova demanda para um determinado produto;
 - Permitir a outras empresas desenvolverem produtos derivados, ou integrações, que a empresa em si não quer desenvolver.
 - Desvantagens:
 - As necessidades dos desenvolvedores externos podem ficar diferentes das dos desenvolvedores internos (caminhos diferentes);
 - Necessidades de estabilidade da aplicação e *backward compatibility* podem frear a criação de novas *features* para uma aplicação.

Casos de uso de APIs

- APIs para desenvolvedores externos primeiro, e internos depois
 - Exemplo: GitHub, começou focado em uma audiência externa, mas com o tempo oportunidades de negócios foram se formando (*developer tools*).
 - Após desenvolvedores *terceiros* começaram a usar uma API GraphQL do GitHub, os desenvolvedores do GitHub também passaram a usá-la.
 - Vantagem: a API pode ser personalizada para atender desenvolvedores externos, em vez de ficar dividida entre dois públicos.
 - Desvantagem: a flexibilidade da API pode acabar gerando gargalos de desempenho.

Casos de uso de APIs

- APIs como produto
 - A própria API é o produto.
 - Exemplo:
 - *Stripe*: fornece APIs para processamento de pagamentos na Internet.
 - *Twilio*: fornece APIs para comunicação via SMS, voz e mensagens.

O que faz uma API ser boa?

- Pontos principais
 - Clareza de propósito, *design* e contexto;
 - Completeza em relação à solução ofertada;
 - Documentação;
 - Usabilidade;
 - Escalabilidade;
 - Desempenho (*performance*).
- É impossível otimizar uma API para todos esses fatores
 - A equipe de implementação precisa tomar decisões difíceis sobre quais são os fatores mais importantes para o usuário final.

Paradigmas de APIs

- Um paradigma de API define a interface que expõe os dados de *backend* de um serviço para outras aplicações.
- Dadas as complexidades que envolvem o desenvolvimento, evolução e manutenção das APIs, elas também têm de ser desenvolvidas considerando-se padrões, protocolos e melhores práticas.
- Os dois principais tipos de paradigma:
 - APIs de Requisição-Resposta (*Request-Response APIs*)
 - APIs Orientadas a Eventos (*Event-driven APIs*)

APIs de Requisição-Resposta

- Tipicamente expõem uma interface através de um servidor web baseado em HTTP.
- Definem um conjunto de *endpoints*.
- Clientes fazem requisições a esses *endpoints* e o servidor retorna as respostas, normalmente como JSON ou XML.
- Os principais paradigmas são:
 - REST;
 - RPC;
 - GraphQL.

APIs de Requisição-Resposta

- *Representational State Transfer* (REST)
 - É o mais popular, e vamos ver com maiores detalhes na próxima aula.

APIs de Requisição-Resposta

- *Remote Procedure Call* (RPC)
 - Enquanto o REST é sobre recursos, o RPC é sobre *ações*. Ele segue duas regras simples:
 - Os *endpoints* contêm o nome da operação a ser executada.
 - Chamadas de API são feitas com o verbo HTTP que é mais apropriado: GET para requisições *read-only* e POST para as demais.
 - Funciona muito bem para APIs que expõem uma variedade de ações que podem ter mais nuances e complicações do que aquelas que podem ser encapsuladas com CRUD.
 - E também para as APIs em existem efeitos colaterais não relacionados ao recurso em questão.
 - APIs RPC também acomodam modelos de recurso complicados ou ações sobre múltiplos tipos de recursos.

APIs de Requisição-Resposta

- *Remote Procedure Call* (RPC)

- Exemplo: Slack's API

```
POST /api/conversations.archive
```

```
HOST slack.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Authorization: Bearer xoxp-1650112-jgc2asDae
```

```
channel=C01234
```

APIs de Requisição-Resposta

- *Remote Procedure Call* (RPC)
 - Exemplo: Slack's API. Alguns métodos:
 - `conversations.archive`: arquiva uma conversa
 - `conversations.close`: encerra uma mensagem direta ou mensagem direta para múltiplas pessoas
 - `conversations.join`: entra em uma conversa existente
 - APIs RPC não são exclusivas para o HTTP.
 - Outros exemplos: [Apache Thrift](#) e [gRPC](#).

APIs de Requisição-Resposta

- GraphQL
 - É uma linguagem de consulta (*query*) para APIs.
 - Foi desenvolvida internamente pelo Facebook em 2012, e liberada publicamente em 2015.
 - Permite aos clientes definirem a estrutura dos dados requisitados, e o servidor retorna exatamente essa estrutura.

APIs de Requisição-Resposta

- GraphQL

Example 2-5. GraphQL query

```
{
  user(login: "saurabhsahni") {
    id
    name
    company
    createdAt
  }
}
```

Example 2-6. Response from GitHub GraphQL API

```
{
  "data": {
    "user": {
      "id": "MDQ6VXNlcjY1MDI5",
      "name": "Saurabh Sahni",
      "company": "Slack",
      "createdAt": "2009-03-19T21:00:06Z"
    }
  }
}
```

APIs de Requisição-Resposta

- GraphQL

- Precisa de apenas um único *endpoint*.
- Não há necessidade de se utilizar diferentes verbos HTTP para descrever uma operação, bastando indicar no JSON se a operação é de *query* ou *mutation*. *Example 2-7. GraphQL API call to GitHub*

```
POST /graphql
HOST api.github.com
Content-Type: application/json
Authorization: bearer 2332dg1acf9f502737d5e
xoxp-16501860787-17163410960-113570727396-7051650
```

```
{
  "query": "query { viewer { login }}"
}
```

APIs de Requisição-Resposta

- GraphQL

- Vantagens

- **Economiza tráfego web:** clientes podem aninhar consultas e buscar dados em vários recursos em uma única requisição.
 - **Evita versionamento:** campos e tipos podem ser adicionados/descontinuados a uma API GraphQL sem afetar as *queries* já existentes.
 - **Payload menor:** como os clientes podem especificar exatamente o que querem, os resultados são previsíveis, sem dados que provavelmente nunca serão utilizados.
 - **Fortemente tipado.**

APIs de Requisição-Resposta

- GraphQL

- Desvantagens

- Adiciona complexidade para o fornecedor da API.
 - O servidor precisa de processamento adicional: para o *parsing* das consultas complexas e verificação de parâmetros.
 - Otimização de desempenho pode ser difícil.

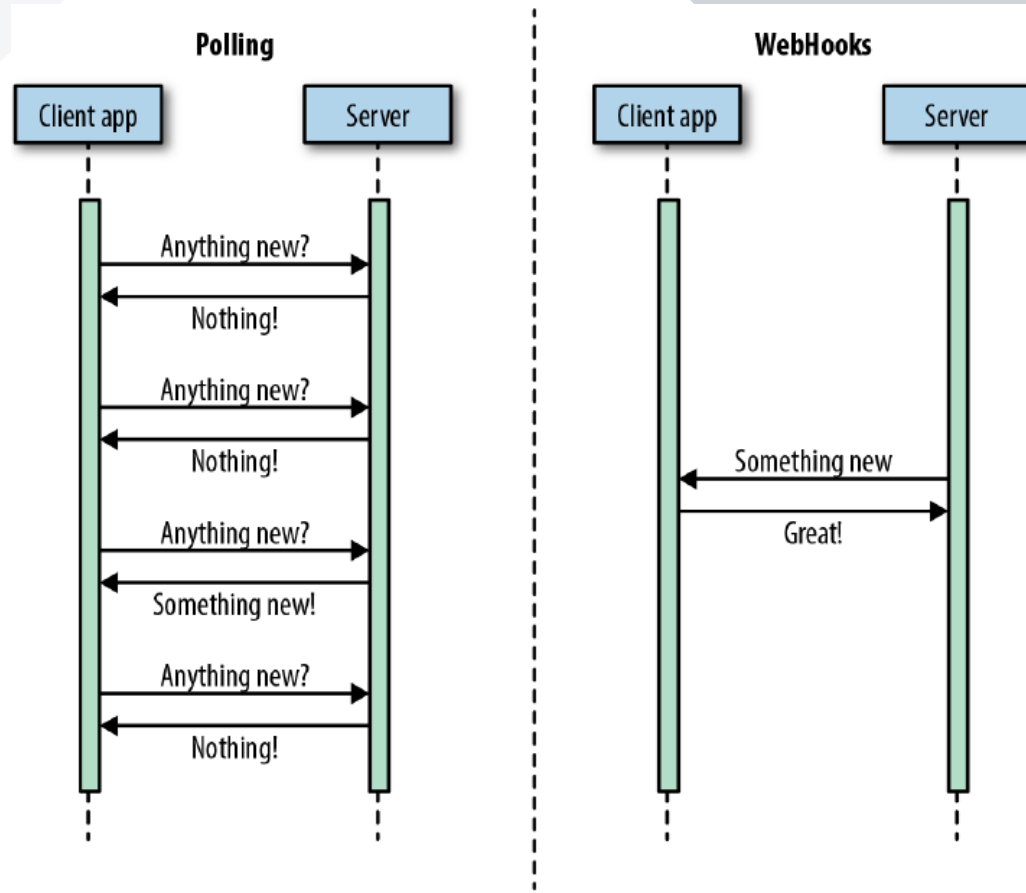
APIs Orientadas a Eventos

- São APIs voltadas a serviços com dados que são modificados com frequência.
- Em vez de ficar fazendo requisições com uma certa frequência, essas APIs compartilham dados sobre eventos em tempo real:
 - *WebHooks*;
 - *WebSockets*;
 - *HTTP Streaming*.

APIs Orientadas a Eventos

- WebHooks

- Consiste em uma URL que aceita um HTTP POST (ou GET, PUT ou DELETE), quando algo acontece. Atualizações ocorrem em tempo real.
- É relativamente fácil de implementar, porque requer apenas a criação de um *endpoint* HTTP para receber eventos.



APIs Orientadas a Eventos

- WebHooks

- Ao mesmo tempo são adicionadas novas complexidades:
 - **Falhas e novas tentativas:** para garantir a entrega.
 - **Segurança:** o ônus de fornecer segurança cai sobre os desenvolvedores de apps, que devem garantir que receberam WebHooks legítimos.
 - **Firewalls:** aplicações protegidas por *firewalls* costumam não conseguirem receber tráfego *inbound*. Nesses casos o uso de WebHooks é muito difícil, senão impossível.
 - **Ruído (*noise*):** normalmente cada chamada WebHook representa um evento único. Quando ocorrem milhares de eventos em um curto período de tempo que precisa ser enviado através de um único WebHook, pode ocorrer ruído.

APIs Orientadas a Eventos

- WebSockets

- Um WebSocket é um protocolo usado para estabelecer um canal de comunicação bidirecional de *streaming* através de uma única conexão TCP.
- Exemplos:
 - O Trello usa WebSockets para enviar modificações feitas por outras pessoas, dos servidores para os navegadores que estão ouvindo um determinado canal.
 - Blockchain usa API WebSocket para enviar notificações em tempo real sobre novas transações e blocos.

APIs Orientadas a Eventos

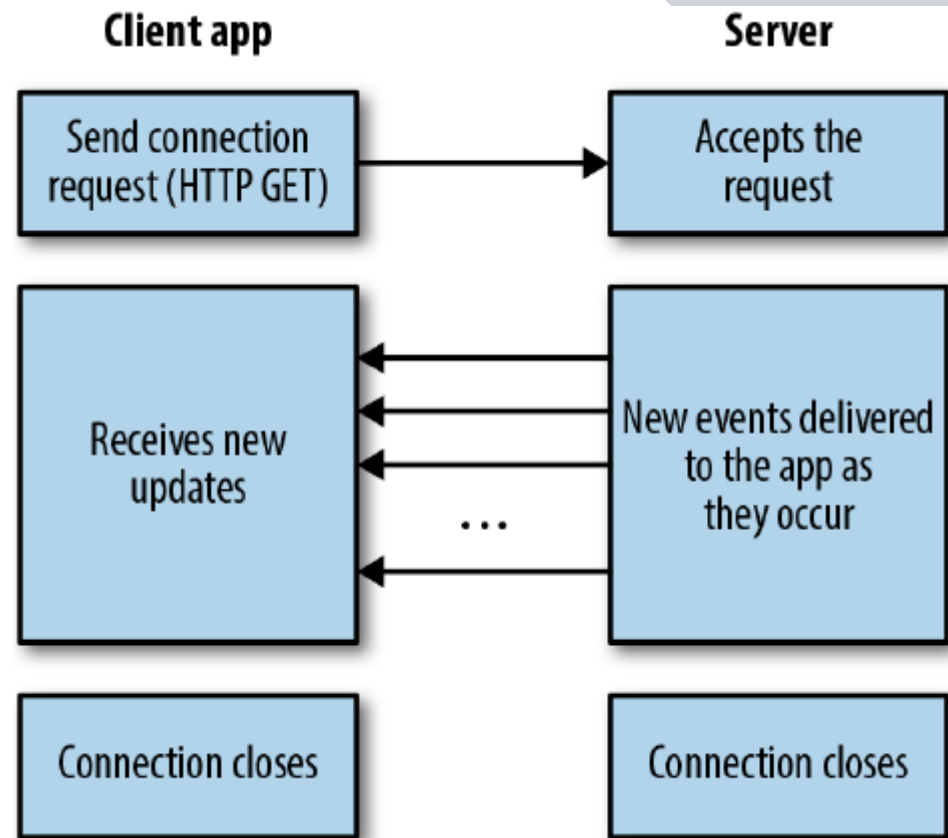
- WebSockets

- Podem permitir comunicação *full-duplex* com um baixo custo.
- Foram projetados para trabalhar sobre as portas 80 ou 443, o que permite que possam ser utilizados, mesmo atrás de *firewalls*.
- São muito bons para conexões rápidas, de *streaming* ao vivo e de longa duração.
 - Nesses casos a Internet precisa ser estável.
- Entretanto, podem sofrer em relação a escalabilidade
 - Se forem abertos 10.000 WebSockets, são 10.000 conexões que um servidor tem de manter.

APIs Orientadas a Eventos

- *HTTP Streaming*

- Em APIs de Requisição-Resposta, as *responses* do servidor têm um tamanho finito.
- Com *HTTP Streaming*, esse tamanho pode ser indefinido, de forma que o servidor continua enviando novos dados em uma única conexão de longa duração aberta pelo cliente.



APIs Orientadas a Eventos

- *HTTP Streaming*

- Duas formas de configurar:
 - Modificar o valor do cabeçalho `Transfer-Encoding` para `chunked`. Assim o cliente indica que os dados chegarão em *chunks* (pedaços) de strings delimitadas por novas linhas.
 - Transmitir (*stream*) dados através de *Server-Sent Events* (SSE). Essa opção é muito boa para clientes consumindo esses eventos em um navegador porque eles podem usar a API padronizada `EventSource`.
- Problema potencial: *buffering* (normalmente limitados, tanto nos clientes quanto nos *proxies*).